

Parallel Fast Fourier Transform

Kunal Goyal 160050026
Aman Bansal 160050028
N. Jagadeep Sai 160050104

November 2019

1 Problem Statement

The Fast Fourier Transform (FFT) is a well-known and widely used tool in many scientific and engineering fields. With the characteristics of intensive computation and being highly divide-and-conquer, FFT is proper to make parallel improvement.

FFT is a fast algorithm to compute discrete Fourier transform (DFT). DFT is defined as

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi k \frac{n}{N}} \quad k = 0, 1, \dots, N-1$$

In this project we implement a double precision 1-D FFT on GPUs using CUDA. We compare the performance of the parallel implementation with a serial one. The implementation details are adopted from [1].

2 Proposed Solution

The Cooley-Tukey [2] algorithm rewrites N as $N = N_1 N_2$, k as $k = N_2 k_1 + k_2$ and n as $n = N_1 n_2 + n_1$, then Equation (1) can be rewritten as

$$X_{N_2 k_1 + k_2} = \sum_{n_1=0}^{N_1-1} \left[e^{-\frac{2\pi i}{N} n_1 k_2} \right] A_{k_2, n_1} e^{-\frac{2\pi i}{N_1} n_1 k_1},$$

where

$$A_{k_2, n_1} = \sum_{n_2=0}^{N_2-1} x_{N_1 n_2 + n_1} e^{-\frac{2\pi i}{N_2} n_2 k_2}.$$

Equation (2) performs N_1 FFTs of size N_2 and N_2 FFTs of size N_1 to compute a N point FFT. If $N_2 = 2$, Equation (2) becomes the radix-2 FFT algorithm, whose data flow diagram is shown in Fig. 2. N point radix-2 FFT can be completed

in $\log(N)$ steps, and then carry out a bit reverse copy. Each step computes N_2 butterfly diagrams. A butterfly diagram is shown in Fig. 3 where ω^k is called the twiddle factor. The computational complexity of radix-2 FFT is $N\log(N)$.

3 Algorithm Implementation

We consider FFTs of size N , where N is the power of 2.

Our implementation contains two parts. One is to compute FFTs of smaller sizes (≤ 512) in one block, the other is to compute FFTs of larger sizes by decomposing a large size FFT into a number of FFTs of smaller sizes using Equation (2).

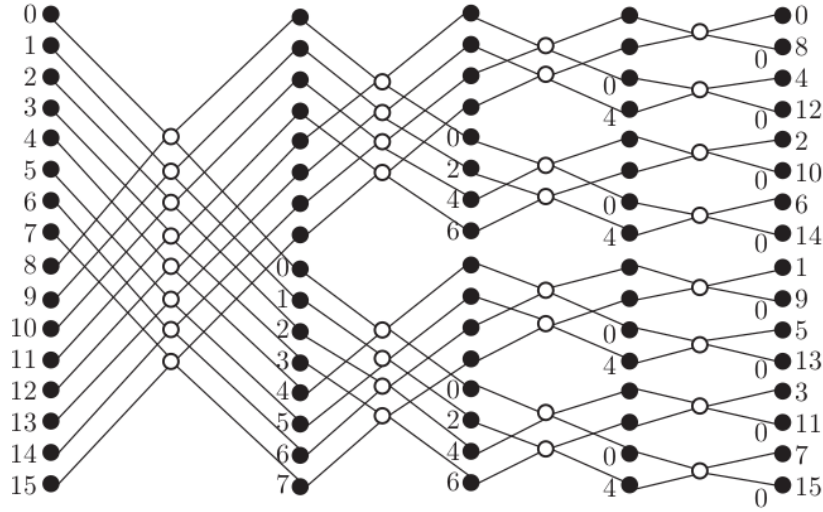


Fig. 2: Radix-2 FFT's data flow diagram

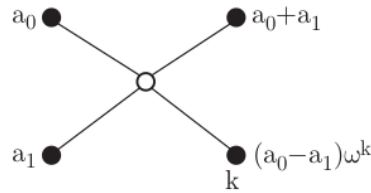


Fig. 3: Butterfly diagram

Pseudo code:

- (1) $m = \log_2 N$
- (2) load data into shared memory
- (3) `--syncthread()`
- (4) *for* $h = m - 1$ *downto* 0
 - a. compute the butterfly
 - b. `--syncthread()`
- (5) read the bit reverse of thread id
- (6) write the data to output buffer

For a larger size (≥ 1024) FFT, we use Equation (2) to decompose a N point FFT into N_1 FFTs of size N_2 and N_2 FFTs of size N_1 , here $N = N_1 N_2$. If the input is a row majored $N_2 \times N_1$ matrix, the Cooley-Tukey algorithm then can be done in four steps as follows.

Pseudo code:

- (1) For each column, compute a N_2 point FFT.
- (2) For each item in the matrix, multiply the twiddle factor.
- (3) For each row, compute a N_1 point FFT.
- (4) Transpose the matrix .

For all $N \geq 1024$, let $m = \log_2 N \geq 10$. There exist $x_1 > 0$, $0 \leq x_2 \leq 3$, such that $4x_1 + 3x_2 = m$, thus $N = 16^{x_1} 8^{x_2}$. We repeatedly use Cooley-Tukey algorithm to decompose large FFT in Step 3 of Cooley-Tukey algorithm into small FFTs, until the size of FFT is 16. The Step 4 of Cooley-Tukey algorithm is transposing the matrix, we develop a kernel to do the last 16 points FFT and all the matrix transpose. The implementation is performed as follows.

- (1) $L=N$
- (2) *for* $i = 1$ *to* $x_1 - 1$
 - a. call kernel `fft-16(L, input)`
 - b. $L=L/16$
- (3) *for* $i = 1$ *to* x_2
 - a. call kernel `fft-8(L, input)`
 - b. $L=L/8$
- (4) call kernel `fft-last-16(L,  $x_1$ ,  $x_2$ , input, output)`

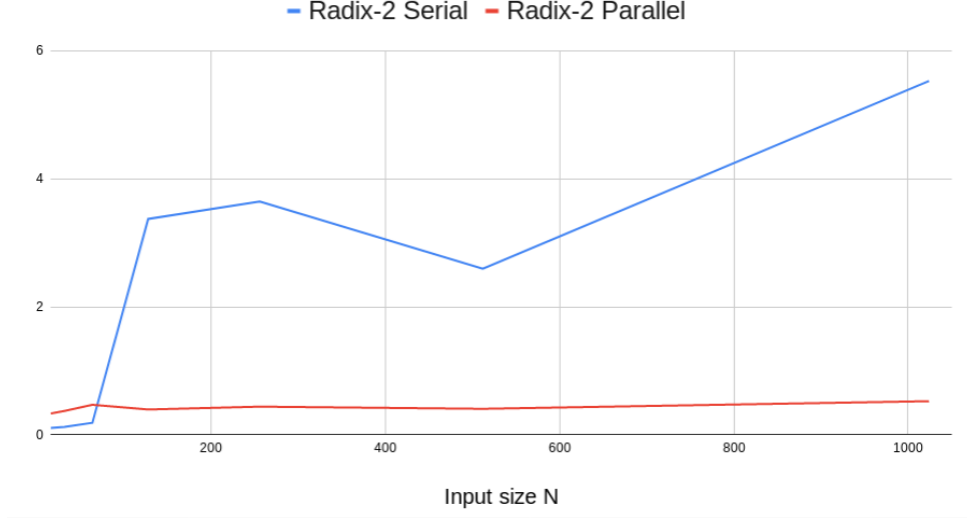


Figure 1: Plot of execution time (ms) for serial and parallel implementations of radix-2 FFT

The code of the implementation can be found [here](#)

4 Results and Observations

The results for the execution times for Radix-2 (for $N \leq 1024$) and Cooley Tukey (for $N \geq 1024$) implementations are in the following tables. The results are for GeForce TITAN X (Pascal) GPU with compute capability 6.1.

Input size N	Radix-2 Serial	Radix-2 Parallel	Speedup
16	0.108	0.333984	0.3233687841
32	0.126	0.374784	0.3361936475
64	0.19	0.470592	0.40374677
128	3.374	0.397184	8.494803416
256	3.648	0.441408	8.26446281
512	2.597	0.407776	6.368692616
1024	5.53	0.52672	10.49893682

It can be observed that for small input size, the serial code is faster than parallel code due to high overhead of parallel thread creation.

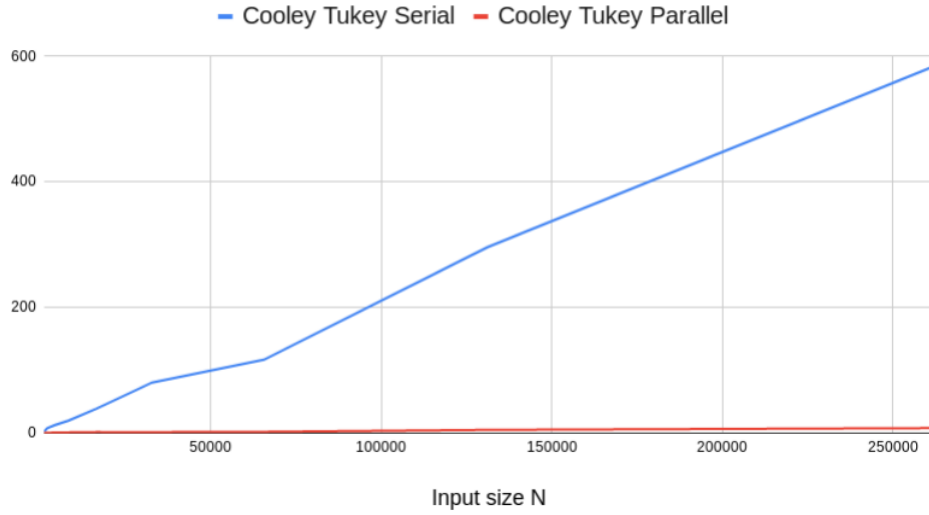


Figure 2: Plot of execution time (ms) for serial and parallel implementations of Cooley-Tukey FFT

Input size N	Cooley Tukey Serial	Cooley Tukey Parallel	Speedup
1024	2.708	0.586848	4.614482796
2048	8.448	0.825632	10.23216154
4096	12.769	1.20912	10.56057298
8192	20.114	1.50179	13.3933506
16384	39.126	2.00883	19.47700901
32768	80.825	1.91325	42.24487129
65536	117.057	2.42531	48.26475791
131072	296.158	5.54144	53.4442311
262144	584.404	8.35088	69.98112774
524288	1.25E+03	1.50E+01	8.30E+01

It can be observed that the speedup is increasing by increasing input sizes due to more utilization of the GPUs.

5 Problems Faced and their Solutions

- In one block of GPU, maximum of only 1024 threads can be executed. Only the threads in a block can share memory. This causes problem in radix-2 implementation as we need to create a lot of threads which shared memory. For the solution to this, we implemented the Cooley-Tukey algorithm which does not need shared memory for its implementation.
- Debugging the CUDA kernel code is a quite difficult task. Since there are many threads, it is difficult to know where the bug is. Writing serial code first and then parallelising helped in minimise bugs and debugging.

References

- [1] Yanjun Liu;Licai Guo;Bin Luo;Xingyi Zhang. An efficient implementation of double precision 1-d fft for gpus using cuda. *JOURNAL OF INFORMATION & COMPUTATIONAL SCIENCE*, 9(2):387, 2012.
- [2] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.